

Serverless Computing: An Overview

In serverless computing, the infrastructure for developing and hosting the application is taken care of by cloud providers on behalf of their customers, the developers. It allows users to write and deploy code without worrying about the infrastructure. This article gives an overview of the serverless framework.



Serverless computing aims to build ‘time-to-value’ code leaving aside other aspects like infrastructure and application services. In this computing model, the focus is on the business logic and delivering code that maps directly to the business value, without worrying about runtime details and management tasks like scaling, scheduling, patching, provisioning, etc.

Both traditional and microservices based architectures can leverage the power of serverless computing.

Serverless applications should have the following characteristics:

- Scale dynamically
- Be stateless
- Allow micro-billing and pay-per-usage
- Be event-driven

How to build a serverless component

To build your application in a serverless environment, the following should be taken into consideration:

- Design your application as functions
- Ensure clear separation between code and environment
- Define the source triggers/events
- Write event-driven code
- Ensure each serverless function does one thing
- Follow version management for maintaining the code
- Deploy code using DevOps methodology
- Integrate plugins and debugging tools as required
- Store results as required in a data sink (database, queue, Amazon S3, etc)

Architectural considerations

While building a serverless component, the following aspects have to be taken into account:

- State(less) management
- Process execution time
- Warmup time
- Version upgrades
- Scaling
- Monitoring, tracing and logging
- Disaster recovery and resiliency
- Coupling of architectural components

Is FaaS (Function as a Service) the same as serverless?

All FaaS components are typically serverless. However, serverless means a larger set of architectural patterns and practices that make extensive use of common services in addition to the